

An Open Source Number Theoretic Transform Accelerator for ML-KEM-512

Brendon Chang

*Dept. of Electrical and Computer Engineering
University of California San Diego
La Jolla, CA, USA
brc016@ucsd.edu*

Marc Boudames

*Dept. of Mathematics
University of California San Diego
La Jolla, CA, USA
mboudames@ucsd.edu*

Abstract—Post-quantum cryptographic schemes, such as ML-KEM, rely on the Number Theoretic Transform (NTT) to make polynomial multiplication in a lattice-based ring efficient enough for practical use. Most published NTT hardware accelerators are built and verified using proprietary, commercially licensed EDA toolchains. These tools cost far more than a student, hobbyist, or low-funded lab can access, significantly limiting who can reproduce or extend the work. This paper describes an ML-KEM-512 NTT and inverse NTT (INTT) accelerator built entirely using an open-source design flow. The design originates from a bit-exact C++ and Python reference model, driving the development of synthesizable Verilog RTL. To prove the viability of the open-source methodology, the accelerator was targeted for a Tiny Tapeout shuttle utilizing the open-source SkyWater 130nm process. Furthermore, the RTL was physically validated on an Arty A7-100T FPGA. Using a custom UART bridge, arbitrary polynomials were streamed from a host machine to the FPGA, executing the forward NTT in 896 clock cycles and the INTT with modular scaling in 1152 cycles. The hardware outputs matched the software reference models with bit-exact precision in real-time, demonstrating that a complete, reproducible path from mathematical reference to verified silicon can be achieved using strictly open-access tools.

Index Terms—post quantum cryptography, ML-KEM, Kyber, number theoretic transform, open source silicon, SKY130, Tiny Tapeout, hardware accelerator

I. INTRODUCTION

Post quantum cryptography (PQC) is a national initiative to develop cryptographic algorithms that are resistant against cryptanalytic attacks from quantum computers, which pose a real threat as they can outperform classical computers in computational power. The sudden reality of quantum computers brute forcing classical cryptographic algorithms has become a major concern for many high-level embedded processors that were designed to be resistant to traditional cryptanalytic attacks. As a result, the National Institute of Standards and Technology (NIST) finalized ML-KEM, a module lattice based key encapsulation mechanism standardized as FIPS 203, as the primary general replacement for RSA and elliptic curve key exchange [1]. Every ML-KEM operation performs polynomial multiplication inside a large cyclotomic ring. Doing that multiplication in the traditional way is quadratic in the polynomial degree. The Number Theoretic Transform (NTT), a ring restricted analogue of the Fast Fourier Transform that stays entirely inside modular integer arithmetic, and turns

that multiplication into an $O(n \log n)$ operation. As a result, the NTT sits directly on the critical path of every ML-KEM encapsulation and decapsulation. Understanding and accelerating it is therefore not an optimization, as it is the core arithmetic of the scheme.

Almost every published NTT accelerator we surveyed while preparing this project was designed and verified with proprietary, commercially licensed EDA tools that can cost tens of thousands of dollars [10]–[12]. Although this is feasible for commercial use, it puts a hard limit on who may use the accelerators. Our goal in this project is to remove such restriction for one specific piece of hardware. We are therefore fabricating an ML-KEM-512 NTT/INTT accelerator using only an open-source process design kit (PDK) and toolchain. This relies on an open PDK (SkyWater SKY130 [3]), an open shuttle and submission framework (Tiny Tapeout [4]), and open simulation tools such as Icarus Verilog and GTKWave. All of our results are publicly available on our github repository [13] such that anyone can verify that our work can be reproduced, checked, and improved upon. These artifacts include the C++/Python reference model for ML-KEM-512, our verilog source modules, testbenches, executable files, and vivado projects where one can emulate our ASIC design on a designated FPGA development board.

In this paper, we will go over some background around the ML-KEM algorithm, how the number theoretic transform and its subsequent inverse transform works from a mathematical perspective, discuss related works surrounding what other ASIC and FPGA NTT accelerators have been developed in the past, how those works differ from our project goals, and finally our design process, implementation, and results of our NTT/INTT engine on the actual Tiny Tapeout toolchain itself.

II. BACKGROUND

A. ML-KEM and Module LWE

ML-KEM (formerly Kyber, now standardized as FIPS 203 [1]) is a key encapsulation mechanism in which security is based on the hardness of the Module Learning With Errors (module LWE) problem over polynomial rings. Its public key and ciphertext operations reduce to sampling polynomials with small coefficients, multiplying them inside a ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, and then compressing and decompressing

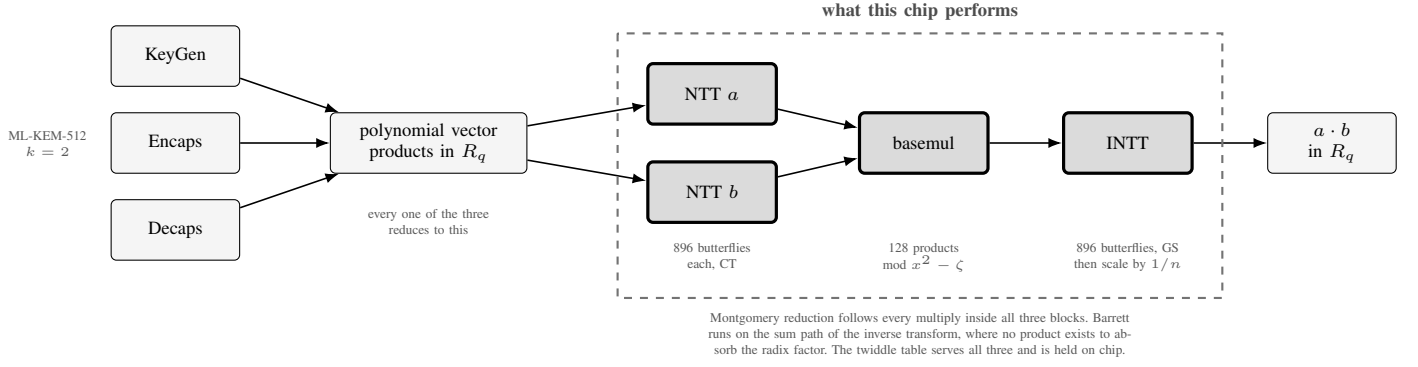


Fig. 1. Where the accelerator sits inside ML-KEM. The dashed boundary marks where our design would exist within a full ML-KEM design. All other blocks exist in software on a host

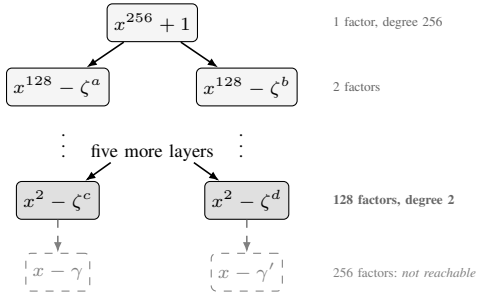


Fig. 2. The incomplete NTT. Each layer of the transform splits every factor in two, and seven layers take $x^{256} + 1$ to 128 quadratics.

for transfer. For ML-KEM-512, the parameter set this project targets, fixes $q = 3329$, $n = 256$, and a module rank $k = 2$. The modulus q , the degree n , and the primitive root used by the transform are all shared across the three standardized ML-KEM parameter sets, with the only parameter changing across security levels is k , the module rank. Thus, hardware built for the transform itself is not tied to the ML-KEM-512 parameter set specifically.

B. The Number Theoretic Transform

Multiplying polynomials in R_q directly costs $O(n^2)$ multiplications. The NTT is based on the same idea as the Fast Fourier Transform (FFT). It decomposes the transform recursively using the Cooley Tukey butterfly [6], but it is restricted to a finite ring $\mathbb{Z}_q$ that contains a root of unity of the required order. Every intermediate value therefore stays an exact integer modulo q , and so we omit floating point rounding from computation. Converting a and b to the NTT domain, multiplying pointwise, and transforming back reduces the multiplication to $O(n \log n)$ modular operations.

A complete decomposition of a negacyclic ring of degree $n = 256$ would require a primitive 512th root of unity, but $q - 1 = 3328 = 2^8 \cdot 13$ contains only 2^8 as a power of two, so $\mathbb{Z}_{3329}$ admits a primitive 256th root and no primitive 512th root. The factorization of $x^{256} + 1$ therefore terminates at 128 quadratic factors rather than 256 linear factors, and the

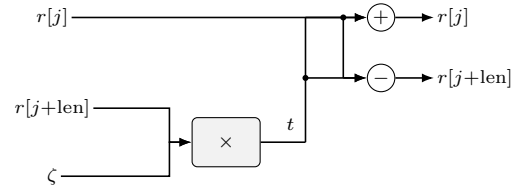


Fig. 3. The forward (Cooley Tukey) butterfly of (1).

transform stops one layer short at seven layers instead of eight. The consequence is that the pointwise stage multiplies degree 1 polynomials modulo those quadratics rather than multiplying scalars.

For ML-KEM-512's $n = 256$, the forward transform decomposes into seven layers, with butterfly lengths 128, 64, 32, 16, 8, 4, and 2, and 128 butterflies per layer, generating 896 total butterfly operations per polynomial and calling 127 distinct twiddle factors.

C. The Forward Butterfly (Cooley Tukey)

The butterfly is the primitive that the whole transform is built from, and such is the operation we commit to silicon. The forward butterfly takes two coefficients and one twiddle factor and produces two coefficients. Within a layer the coefficients are paired at a fixed distance len , and each pair $(r[j], r[j + \text{len}])$ is combined as

$$\begin{aligned} t &\leftarrow \zeta \cdot r[j + \text{len}], \\ r[j + \text{len}] &\leftarrow r[j] - t, \\ r[j] &\leftarrow r[j] + t, \end{aligned} \quad (1)$$

where ζ is the twiddle factor for the current block. Reading in order, (1) multiplies the upper coefficient by the twiddle to form t , then adds t to the lower coefficient for one output and subtracts it for the other. The cost is one modular multiply, one add, and one subtract, computed in place because the two outputs overwrite the two inputs they came from. Structurally, the two outputs are the same pair of inputs combined under $+t$ and $-t$, which is why the pair of outputs carries exactly the

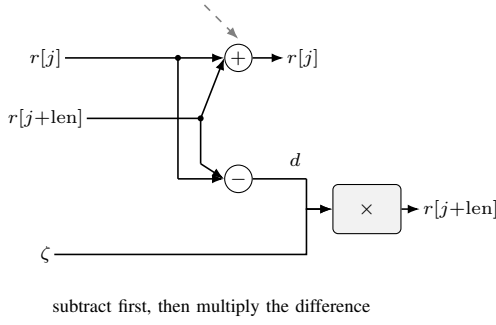


Fig. 4. The inverse (Gentleman Sande) butterfly of (2). The order is reversed against Fig. 3. There is no multiplication on this path, so the growth is bounded by Barrett reduction

information the pair of inputs did, thus making the transform invertible.

D. The Inverse Butterfly (Gentleman Sande)

The Inverse Number Theoretic Transform recombines the 128 quadratic factors back into a single polynomial, and also mirrors the seven layer structure of the forward transform rather than compensating for it. It walks through the layers in the opposite order with the twiddle index descending rather than ascending, but requires a final $1/n$ scaling pass. The inverse butterfly [7] runs the same relationship backwards to regenerate two half results into one, with the main difference being the order of the operations:

$$\begin{aligned} t &\leftarrow r[j], \\ r[j] &\leftarrow r[j] + r[j + \text{len}], \\ r[j + \text{len}] &\leftarrow \zeta \cdot (r[j + \text{len}] - t). \end{aligned} \quad (2)$$

The forward form multiplies first and then adds and subtracts, but the inverse form adds and subtracts first and multiplies the difference afterwards. Both outputs are computed from the original values of $r[j]$ and $r[j + \text{len}]$, so this is again a read then write operation on the same two locations.

Two properties of this pair follow directly, and both are used later. First, the two forms are the same primitive sequenced differently, so a single arithmetic core with a mode select covers both directions. Second, the sum path in (2) is a plain addition with no reduction attached, so running sums can grow out of range across layers. A reduction is therefore required on that path to return the value to a centered representative.

E. Modular Reduction: Montgomery and Barrett

Every NTT butterfly performs a modular multiply, and a modulo operation, and like integer division, is one of the most expensive primitives to implement in hardware. Montgomery reduction [8] expresses values in a shifted radix, where $R = 2^{16}$ for this design's 16 bit coefficients. This means that reduction modulo q becomes a multiply, a truncation, and a subtract to remove division. Montgomery reduction does not return $a \bmod q$ but $aR^{-1} \bmod q$, so the twiddle table is stored pre-multiplied by R and the residual factor cancels. Barrett reduction [9] achieves a similar effect for reducing a value

back to a centered representative without Montgomery's radix change, and the the reduction the inverse transform requires on the sum path of (2).

The two reductions are complementary rather than interchangeable, and ML-KEM uses each where it fits. Montgomery suits the butterfly multiply because its domain is preserved by multiplication, so the radix factor is absorbed by the product. Barrett suits sums, which carry no product that could absorb that factor, and this is why the inverse transform needs it on the sum path of (2). Both are standard techniques in modular arithmetic hardware, but a hardware translation must still respect their bit width restrictions exactly. Errors here are the most common source of subtle bugs in NTT hardware, and they are difficult to diagnose after the fact, which is why we began from a software reference model.

F. Loop Structure and Independence

The transform is three nested loops. The outer loop halves the butterfly length, the middle loop steps across blocks and draws the next twiddle, and the inner loop walks the pairs within a block. The two consequences of that structure matter for any hardware translation. Since hardware has no loops, the loop variables must become state, and the loop increment and reset rules must become the transitions that advance that state. Additionally, the butterfly reads and writes the same two locations, so both reads must be completed before either write.

The loop structure also has an independence property. Indeed, within a single layer, every butterfly touches the pair of addresses $(j, j + \text{len})$, and within that layer the pairs are disjoint, so each address is read exactly once and written exactly once. No butterfly within a layer can read a value that another is still writing, and the only ordering constraint in the whole transform is between layers.

G. Open Source Silicon: SKY130 and Tiny Tapeout

SkyWater SKY130 [3] is a 130 nm process whose design kit is published rather than licensed. The standard cell libraries, the device models, and the design rules are all openly available, which means the entire path from Verilog to mask geometry can be walked with tools that carry no license cost. This path is called LibreLane [5], an open flow from RTL to GDSII that performs synthesis, floor planning, placement, clock tree synthesis, routing, static timing analysis, and signoff, and its output is the GDSII file that the foundry converts into photomasks. The relevance to this project is that every number we report in Section V comes from a flow a reader can install and rerun.

Tiny Tapeout [4] is a shuttle built on an open-source software that allows users to design their own RTL code and use open-source process design kits (PDK) to create their own design on a shuttle. It places many small designs onto a single die and shares the mask cost across all of them, which reduces the price of fabricating a design from six figures to roughly the cost of a textbook. A harness multiplexes the projects so that one project is enabled at a time, and each project is presented to that harness through a fixed template. To submit, simply pull

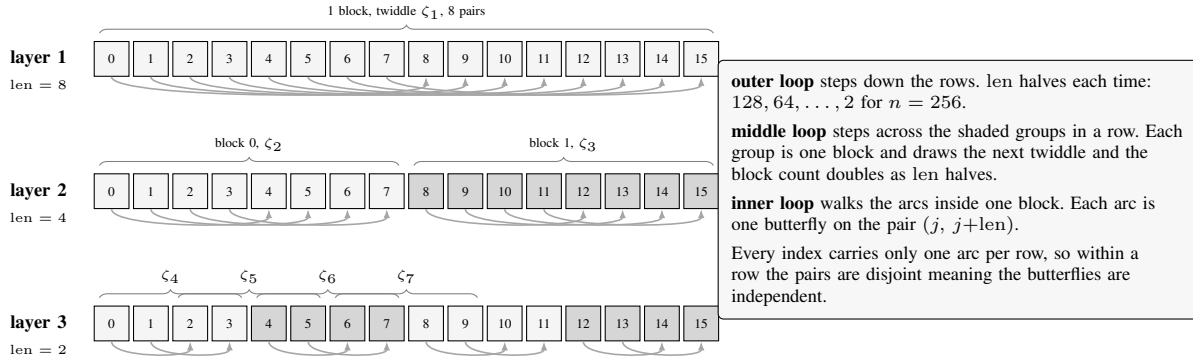


Fig. 5. For ML-KEM-512 the same pattern runs over 256 points and 7 rows, with len halving from 128 down to 2, the block count doubling from 1 to 64, and 128 iterations in every row for 896 butterflies in total.

request against the shuttle repository, with passing requiring the design to harden, pass the shuttle’s own precheck, and pass its testbenches on the post layout netlist.

This template imposes three constraints, and it is important to note them here as IV follows. First, the pin budget is fixed at twenty four signals, of which eight are input only, eight are output only, and eight are bidirectional. Second, the flow offers no macro SRAM, so any array declared in the RTL is synthesized into registers and their read multiplexers. Third, area is quantized into tiles, and a design must declare how many tiles it occupies and fit within them, where the largest configuration available is 8×2 . Lastly, the clock is supplied by the demonstration board and is specified up to 50 MHz, which is the frequency we target.

III. RELATED WORKS

Hardware acceleration of polynomial arithmetic for lattice cryptography is an already active domain, and the designs in it, trade area against latency in fairly predictable ways. We group the prior work by the platform it targets and extract from such the costs a designer can actually see.

A. FPGA Accelerators

Most published ML-KEM NTT accelerators target FPGAs. Taghavi et al. [12] present KNightCore, a unified constant geometry datapath that executes the forward NTT, the inverse NTT, and pointwise multiplication on a single pipelined butterfly unit. Their main contribution is a pointwise multiplication method that reuses the twiddle factor ROM already present for the transform, thus removing the use of dedicated multipliers and storage. On a Xilinx Artix-7 they report 650 LUTs, 130 registers, 1.5 BRAMs, zero DSP blocks, and 0.058 W, completing NTT or inverse NTT in $7 \times (128 + 4) = 924$ clock cycles. Barrett reduction is used throughout, and a ping pong dual port BRAM arrangement keeps the pipelined.

The wider FPGA literature that KNightCore surveys spans roughly in both directions. Designs built for speed reach NTT latencies near 1.0 to 1.5 μ s using two to five DSP slices and up to ten BRAMs. Designs built for area reduce or remove DSPs and accept longer latency instead, and two examples make the range clear. The first implementation without DSPs uses 541

LUTs and four BRAMs at 1.10 μ s, while the other uses 676 LUTs and 633 registers at 6.55 μ s, but the scoreboard stays constant through out both designs. FPGA designers optimize against LUT, DSP, and BRAM counts, and DSP usage is the number most often placed in the abstract.

B. ASIC and SoC Implementations

Far fewer NTT accelerators reach fabricated silicon. Dam et al. [10] report the closest comparison point to our work. This included the integration of a dual butterfly unit NTT accelerator into a 64 bit RISC-V CPU through the Rocket Custom Coprocessor interface, then fabrication of the complete SoC in a 180 nm CMOS process. Their accelerator completes a forward NTT in 1514 clock cycles and an inverse NTT in 1413 clock cycles, for an area overhead of 8.7% over the unaccelerated CPU. The finished chip occupies 3.8025 mm² (297 k gate equivalents), reaches 118 MHz at 2.0 V, and draws 307.9 mW at that frequency, with a minimum of 5.913 μ W at 10 kHz and 0.9 V. They also make an observation we want to repeat here, which is that most comparable systems are evaluated on FPGAs or through synthesis alone, and that measurements taken from a physical chip remain rare in this space.

Kundi et al. [11] take a different direction and target a single NTT datapath that supports both ML-KEM and ML-DSA, thus includes the handling of two different moduli and two different transform sizes in the same hardware.

C. How Our Work Differs

We differ from the prior work in three ways, and none of them is a performance claim.

Beginning with the design target, the works above optimize latency, LUT count, or DSP count against a device that is either reconfigurable or fabricated on a commercial process at a cost that places it outside most academic budgets. We aimed at a fixed silicon area on a shared shuttle instead. The binding constraints there are a 24 pin budget, the absence of any SRAM macro, and a quantized tile grid, and those three shaped the architecture far more than any throughput target did. Section V reports what each one cost us.

We also carry both Montgomery and Barrett reduction on the same datapath, which the prior designs do not. KNightCore uses Barrett throughout, and most FPGA designs pick one reduction and apply it everywhere. We implement each where the arithmetic calls for it, Montgomery after every multiply and Barrett on the inverse transform’s sum path, and we verify that the two are separate units rather than one standing in for the other, so, in exchange, an inverse butterfly needs one multiply instead of two.

The last difference is reproducibility, which is the emphasis of this project. Dam et al. correctly observe that physical chip measurements are scarce, and their 180 nm results come from a commercial process with a proprietary design kit. Our entire flow runs on the open SKY130 PDK with LibreLane, so every number in Section V comes from a tool a reader can install and rerun against our published RTL.

Yet, there are places where we do not compete. These include: Dam et al. report a complete SoC while we report an arithmetic engine, so their area figure includes a RISC-V CPU that ours does not, KNightCore holds the polynomial in on chip block RAM and we do not, for reasons given in Section V. Thus, comparing numbers straight across those boundaries would mislead, so we avoid it.

IV. METHODS

A. Reference Model

We began in software, for any error in the reduction bit widths produces hardware that simulates cleanly and computes the wrong answer, so building the reference first gave us a golden model that did not depend on any logic in RTL.

The model is a bit exact C++ reimplement of the full ML-KEM-512 pipeline, with the NTT, the inverse NTT, and the modular reduction kernels isolated as independently testable units. It works in `int16_t` throughout, which is the same representation the RTL later used. Thus with the two in the same arithmetic type from the start implies that a disagreement between is always a logic difference and never a representation difference.

Then we validated this model at three levels: the transform alone was checked for the round trip identity, the complete KEM was checked for agreement between encapsulation and decapsulation, and the whole implementation was checked byte exact against the NIST ACVP test vectors [2]. This was crucial because passing ACVP established that our arithmetic conventions are the standard’s conventions and not merely self consistent ones.

Alongside the C++ we wrote a bit exact Python mirror, which generates the vectors the RTL testbenches consume. A worked example script prints the same base cases in decimal, hex, and binary, so that any single butterfly can be checked by hand when a waveform disagrees with expectation.

One further check compares the model against nothing but the mathematics. The incomplete NTT is a Chinese Remainder Theorem map, so $\text{NTT}(f)_{2i}$ and $\text{NTT}(f)_{2i+1}$ must equal f reduced modulo $x^2 - \zeta^{2 \cdot \text{brv}_7(i)+1}$. We test the transform against

that definition directly, without the use of layers, twiddle ordering, or any butterfly.

B. RTL Translation and Verification

The next step involved translating the reference into synthesizable Verilog. We kept the module boundaries aligned with the software, so that each RTL block had a single software function to be checked against. The result includes Montgomery and Barrett reduction, the modular multiplier, the Cooley Tukey and Gentleman Sande butterflies, a 128 entry twiddle factor ROM in Montgomery form, and the control logic that sequences all of it.

Every module was verified bit for bit against vectors emitted by the C++ and Python models. The test benches cover the butterfly in isolation, the forward and inverse engines, the round trip identity $\text{INTT}(\text{NTT}(f)) = f$, and the full polynomial multiply against schoolbook calculations.

Comparing hardware against our own model has an obvious weakness, which is that both could be wrong in the same way, thus four checks in the suite therefore avoid the model entirely. We first check that $\text{NTT}(1)$ is $(1, 0)$ at all 128 quadratic factors, a result that follows from the definition because a constant polynomial is already its own remainder. Then we check that the Gentleman Sande butterfly undoes the Cooley Tukey butterfly and returns $2a \bmod q$, following then recompute every ROM entry from $\zeta^{\text{brv}_7(k)} R \bmod q$ inside the test rather than trusting the generator that wrote it. Lastly, we require the two reduction units to return different results for the same input, related by exactly R^{-1} , which fails immediately if one has been quietly implemented in terms of the other.

The Barrett reducer was tested extensively. The input is a single 16 bit word, so all 65,536 values are reachable in a short test, and the theorem it implements is stated for $|a| < 2^{15}$, in which such bound puts $a = -32768$ exactly on the boundary, so we check that value rather than assume it is trivial.

C. FPGA Demonstration

The live demonstration performs the iterative NTT engine on an FPGA, meaning the single Cooley Tukey butterfly, the twiddle ROM, the coefficient memory, and the control FSM. It exercises the same arithmetic RTL that we submitted for tapeout, so the on board behavior previews the fabricated silicon.

Three modes run on the board. The first is self checking and needs no host at all. We preload a set of known input polynomials, together with their golden NTT outputs precomputed by the Python mirror, into block RAM, in which a button press transforms the input in place, a comparator comes back against the stored golden output, and an LED reports pass or fail.

The second mode accepts the arbitrary input. A UART bridge streams 256 coefficient polynomials from a laptop into the engine, and the transformed coefficients are streamed back and compared on the host against the C++ and Python reference in real time. Since the input is supplied at run time,

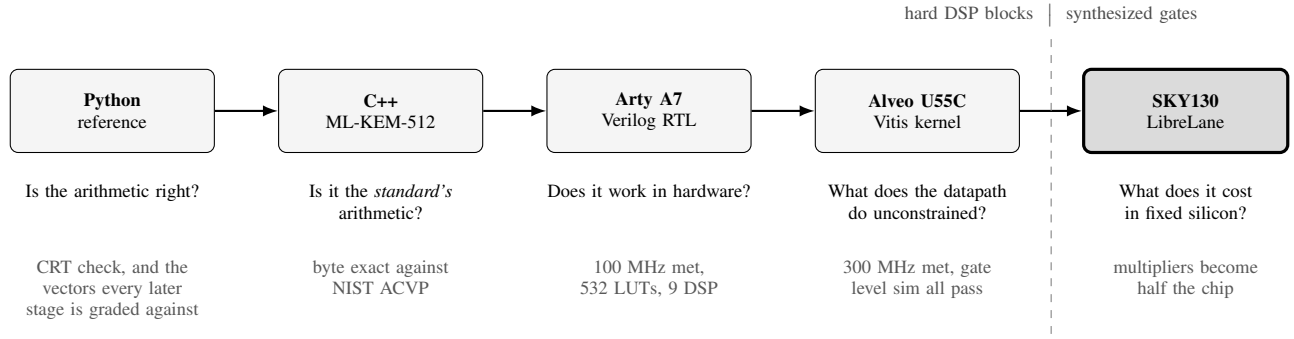


Fig. 6. The five stages. Multipliers are hard DSP blocks on both FPGAs and synthesized gates on SKY130, and neither area nor timing closure transferred across it.

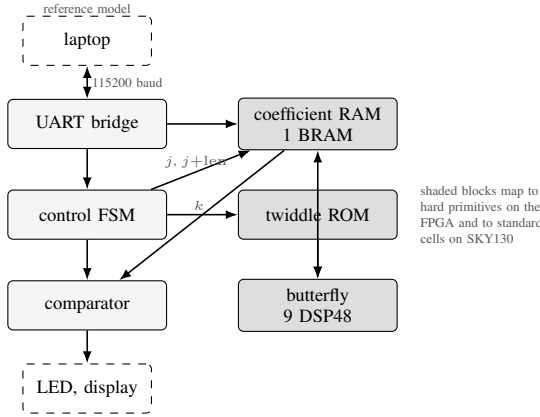


Fig. 7. FPGA demo schematic

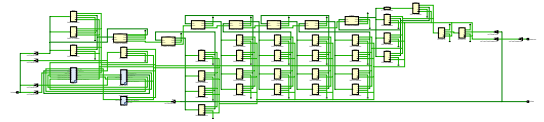


Fig. 9. Vivado post synthesis schematic of the Arty A7-100T top level. This is the board interface, not the datapath: clock buffers, the two stage reset synchroniser, the button debouncer and pulse stretcher behind the pass and fail LED, and the heartbeat counter. The NTT engine is the block they connect to. Included for structure rather than for reading, since a schematic of this size is not legible at column width.

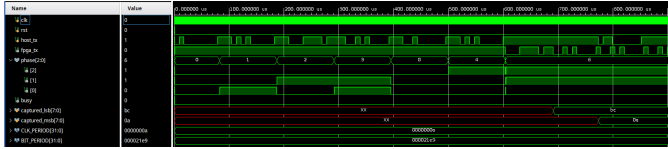


Fig. 8. Simulation waveform from the Arty A7-100T build.

an observer can enter their own polynomial on the spot, which makes the demonstration interactive rather than canned.

The board itself contains only LEDs for human display verification. A single LED reports pass or fail, and a second one blinks while the engine is running, which is enough to show that a transform started and finished. Anything finer than that is read on the host, where the Python driver prints the transformed coefficients alongside the reference and flags the first index that disagrees.

We built the demonstration on two devices. The Arty A7-100T carries the standalone engine described above. We also packaged the same arithmetic as a Vitis kernel on an Alveo U55C, to measure what the datapath does when the device and the clock are not the limit.

D. Path to Tapeout

The FPGA builds informed us the arithmetic was emulated correctly in hardware, but they did not tell us what it would cost in fixed silicon, and the two answers turned out to be quite different. On an FPGA the multipliers are hard DSP blocks that consume 5% of an Arty and 0.13% of a U55C. Synthesized into SKY130 standard cells, the same arithmetic became roughly half the chip.

Therefore, we restructured the design around the three shuttle constraints listed in Section II, where three changes were made.

First, we moved the 256 coefficient polynomial off chip, since a 256 by 16 bit array is 4096 registers, and with no SRAM macro available the flow synthesizes that into registers plus two 256 to 1 read multiplexers. We measured a build that captured the array at 21,248 cells for the memory alone against 3458 cells for everything else, which is 86% of the design. Thus, the host now holds the polynomial and walks the address pattern, and the chip performs the arithmetic.

TABLE I
FPGA RESOURCE USAGE.

Resource	Arty A7-100T		Alveo U55C	
	used	of	used	of
LUTs	532	63.4 k	619	1.30 M
Registers	284	126.8 k	215	2.61 M
DSP slices	9	240	12	9024
Block RAM	0.5	135	0.5	2016
Occupancy	0.84% of LUTs 3.75% of DSPs		0.05% of LUTs 0.13% of DSPs	

TABLE II
FPGA POST ROUTE TIMING AND POWER.

	Arty A7-100T	Alveo U55C
Device	xc7a100tcsq324-1	xcu55c-fsvh2892-2L-e
Clock	100 MHz	300 MHz (kernel)
Worst setup slack	+0.960 ns	+0.989 ns
Failing endpoints	0 of 922	0 of 264
Worst hold slack	+0.176 ns	+0.009 ns
Total power	0.119 W	3.318 W
dynamic	0.022 W	0.044 W
static	0.097 W	3.273 W

Next, we kept the twiddle table on chip. The table is 128 constants, and a constant is a fixed function of its index rather than mutable state, so the synthesizer minimizes 1536 bits of ROM into roughly 463 cells without the use of registers.

Lastly, we pipelined the modular multiplier. If written combinationally, the Montgomery path connects three multipliers in series (the 16 by 16 product, then the multiply by $q^{-1} \bmod R$, then the multiply by q), which is roughly 120 gate levels. We cut it into three stages with one multiply in each with the same reasoning later applied to the Barrett reducer, and Section V describes what post layout timing analysis told us about both.

V. RESULTS

A. FPGA Implementation

Table I reports what the design consumes on each FPGA and Table II reports what it achieved. Both designs met timing with no failing endpoints.

The logic is comparable on both devices, 532 LUTs against 619 and 9 DSP slices against 12, where the differences come from the two device families mapping the same multipliers onto different DSP primitives. The three fold clock increase therefore came from the device and its speed grade rather than from any changes we made. Static power also dominates on both parts, with the accelerator's own switching costs 22 mW on the Arty and 44 mW on the Alveo, while the devices hosting it draw 97 mW and 3273 mW respectively while doing nothing.

We also ran a gate level simulation of the synthesized Alveo netlist rather than the RTL. It completed a full 256 coefficient transform in 16,133 cycles with all 256 coefficients correct,

TABLE III
IMPLEMENTATION SUMMARY, SKY130.

Technology	SkyWater SKY130, 130 nm
Standard cell library	sky130_fd_sc_hd
Operating voltage	1.80 V nominal
Die area	$334.88 \times 225.76 \mu\text{m}$ (2 by 2 tile)
Core area	$72,565 \mu\text{m}^2$
Logic area	$52,731 \mu\text{m}^2$
Gate count	14,048 GE
Cell count	6067
Core utilization	75.5%
Target clock	50 MHz
Maximum frequency	55.4 MHz at the slow corner
Worst setup slack	+1.955 ns, 0 violations
Worst hold slack	+0.106 ns, 0 violations
Routed wire length	128,063 μm
Longest net	335.4 μm
Routed nets	5023
Vias	36,218
Total power	2.623 mW
Worst IR drop	$8.24 \times 10^{-5} \text{ V}$

TABLE IV
SKY130 PLACE AND ROUTE RESULTS.

Standard cell area	$52,731 \mu\text{m}^2$
Cell count	6067
Die area	$75,603 \mu\text{m}^2$ (2 by 2 tile)
Core utilization	75.5%
Target clock	50 MHz
Worst setup slack	+1.955 ns, 0 violations
Worst hold slack	+0.106 ns, 0 violations
DRC errors	0
LVS errors	0
Antenna violations	0
Inferred latches	0
Total power	2.623 mW
internal	1.591 mW
switching	1.032 mW
leakage	0.0001 mW
Worst IR drop	$8.24 \times 10^{-5} \text{ V}$

thus showing the DSP and BRAM primitives the tool actually utilized.

B. ASIC Implementation

Table IV reports the LibreLane results for the current build on SKY130. The design occupies a 2 by 2 tile.

Table V breaks the cell count down by module. The arithmetic accounts for 75% of the design, which is the inverse of the FPGA result where the multipliers were free hardened blocks.

C. Timing Closure

We hit setup timing violations three times, in which every diagnosis came from reading the static timing report rather than from inspecting the RTL. The third one is the interesting case, because the first two fixes were aimed at the wrong thing.

The first violation appeared in an early build at -0.970 ns across four endpoints. The failing path ran through a 32 bit accumulator and up its ripple carry chain, because the

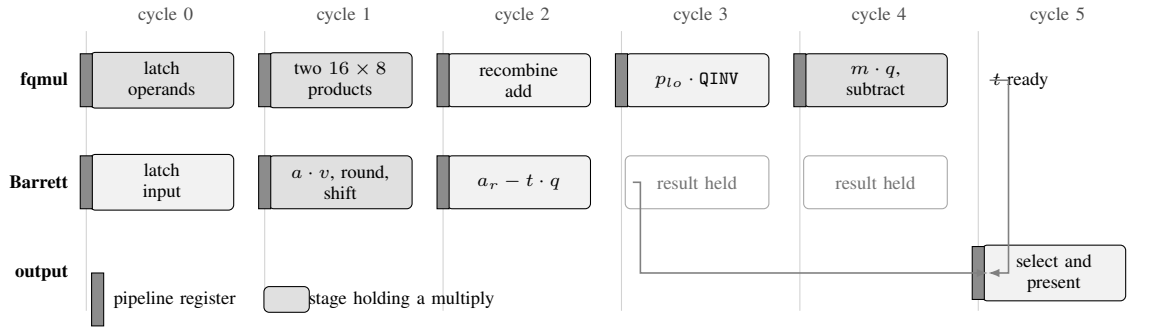


Fig. 10. Stage boundaries in the datapath. Every stage holds at most one multiply, and no register drives both a long path and a short one. The second rule is what two failed builds taught us, since a register feeding both gets padded for hold and the long path pays. Barrett finishes in three cycles against the multiplier's five, so its result is held until the outputs are read.

TABLE V
SYNTHESIS CELL COUNT BY MODULE.

Module	Cells	Share
Multiplier and Montgomery reduction	2318	47.8%
Barrett reduction	1182	24.4%
Butterfly operand and output mux	712	14.7%
Twiddle factor ROM	463	9.6%
Byte serial interface	172	3.5%
Total	4847	



Fig. 11. The routed die seen through its stacked metal layers. The die is 334.88 by 225.76 μm , a 2 by 2 shuttle tile, holding 6067 standard cells at 75.5 percent core utilization with 128,063 μm of routed wire across 5023 nets

serial multiplier we used at that point shifted the multiplicand left into a full width accumulator. By restructuring into the shift right form, the accumulator holds only the top half of the product and the multiplier register doubles as the bottom half, thus narrowing the adder from 32 bits to 17 and moved the slack to +6.197 ns. Cell area fell as well, from 24,971 to 22,804 μm^2 , because a shorter path needed fewer repair buffers.

The second violation appeared after we added the Barrett unit, at -1.511 ns across 58 endpoints, with every failing path beginning at the same register, the operation select, and

thus ending in the result shift register. We had left the Barrett reducer combinational so that the two operations that do not need a multiplier could answer in zero clocks. As the operation select chooses Barrett's input, that decision placed the entire reduction inside a single cycle path, where reading the per cell delays showed the 21 ns of logic splitting into roughly 5.0 ns of control fanout, 13.5 ns for the reduction itself, and 2.5 ns for the output multiplexer.

This breakdown changed the fix, for registering only the reducer's output, which is the obvious repair, removes the last 2.5 ns and leaves about 18.5 ns against 20.82 ns required, a margin of roughly 2 ns. Thus we placed the reducer between two registers instead, one ahead of it to separate the control fanout and one behind it to separate the output multiplexer, which leaves the 13.5 ns reduction a full clock cycle to itself. The cost is 32 registers and an increase of 0.7% in cell count. The extra cycle is free, because the multiplier already takes three, so every operation now takes three clocks, which also removed a special case from the front end.

That build did not close either. It came back at -1.838 ns across 33 endpoints, so the violation count fell from 58 and the total negative slack from -19.30 ns to -12.20 ns, but the worst path was slower than it had been before. A fix that lowers the count while raising the worst slack is a sign that the diagnosis is wrong, so we read the per cell delays again, this time on the path that actually failed rather than the one we expected, and found four delay buffers sitting on the reducer's own input register, contributing 4.18 ns before any gate of arithmetic ran. The netlist held 227 of them. They are hold repair, and the placer inserts them because that register drives a long path and a short path at the same time, so padding the source net is the only way it can protect the short one.

The rule that follows is that no register may drive both a long path and a short path, and Barrett broke that rule in its own definition, since $c = a - t \cdot q$ uses a through both multiplies and then again directly in the final subtract. We split the reducer so that stage one forms the quotient and carries a in a register of its own, and stage two finishes. We also narrowed t from 16 bits to 6, which it can be since $|t| \leq 10$, and that turns $t \cdot q$ into a 6 by 12 constant multiply instead of a 16 by

TABLE VI
STANDARD CELL USAGE BY CATEGORY.

Category	Count	Share
Combinational logic	1519	25.0%
Tap	1037	17.1%
NOR	890	14.7%
NAND	640	10.5%
OR and XOR	541	8.9%
AND	440	7.3%
Flip flops	327	5.4%
Clock tree	317	5.2%
Multiplexer	93	1.5%
Miscellaneous	87	1.4%
Inverter	86	1.4%
Buffer	85	1.4%
Diode	5	0.1%
Total	6067	
Fill and decap	6454	not counted above
All placed instances	12,521	

TABLE VII
KEY METRICS MEASURED ACROSS THE ARTY, ALVEO, AND SKY130
IMPLEMENTATION

	Arty A7	Alveo U55C	SKY130
Clock	100 MHz	300 MHz	50 MHz
Logic	532 LUTs	619 LUTs	6067 cells
Multipliers	9 DSP	12 DSP	synthesized
Total power	119 mW	3318 mW	2.62 mW
Static power	97 mW	3273 mW	0.0001 mW
Power vs ASIC	45×	1265×	1×

12 one. We gave the multiplier its own operand register for the same reason, so that the array starts from registers feeding nothing else, and we split the 16 by 16 product into two 16 by 8 partial products and a recombine, which took that stage from 63 levels to 46.

This build closed and met the 50 MHz with +1.955 ns of slack at the slow corner and had no violations at any of the nine corners the flow reports, with hold at +0.106 ns and the DRC, LVS, antenna and latch counts all zero. Latency rose from three clocks to five, which the byte serial interface hides, since it needs four write cycles and four read cycles per operation anyway. Power fell from 4.58 mW to 2.62 mW, which we did not predict, and the reason is that five shallow stages switch less than three deep ones.

D. Cross Platform Comparison

Table VII places the three implementations of our design side by side, where the comparison is useful mainly for what it says about where cost appears.

The power difference is dominated by static draw. An FPGA pays the leakage of an entire device regardless of how much of it the design uses, and the U55C draws 3.27 W standing still for a kernel that occupies 0.05% of its fabric. One caveat belongs with this comparison. The three platforms run at different clocks, and the ASIC moves its operands over an 8 bit bus while the FPGA builds hold the polynomial locally,

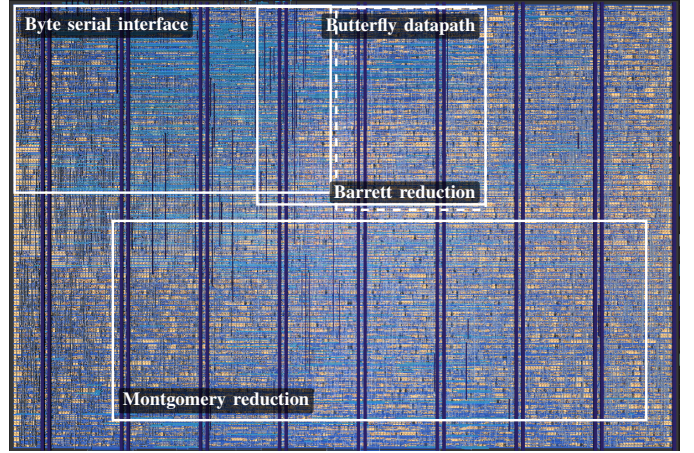


Fig. 12. Block locations on the routed die. Montgomery holds the lower half of the die on its own, Part of Barrett is contained within the butterfly, where the RTL instantiates it, and the twiddle ROM is absent because it is combinational and has no flip flops to locate

so this measures standing cost per unit of deployed function and not energy per transform.

FPGA timing closure also turned out to be a poor predictor of ASIC timing closure. The design met 100 MHz on the Arty and 300 MHz on the Alveo with a combinational multiplier structure, and that same structure could not meet 50 MHz on SKY130.

E. Comparison with Prior Work

Table VIII places our result next to the two closest published designs. We report it with the boundaries stated, because the three designs do not implement the same scope.

The area figures are not directly comparable. Dam et al. report a complete SoC that includes a 64 bit RISC-V CPU, so their 3.8 mm² covers considerably more than an NTT engine, and their 8.7% area overhead figure is the more meaningful number for the accelerator alone and KNightCore reports FPGA primitives rather than silicon area. Our figure is the arithmetic engine only, and it excludes the coefficient storage that both of the other designs include.

The power figures carry the same caveat and one more. Our 2.62 mW is measured at 50 MHz on a 130 nm process, while Dam et al. measure 307.9 mW at 118 MHz on 180 nm and report 5.913 μ W at 10 kHz, so their design spans a far wider operating range than we characterized.

What the table does show cleanly is the difference in scope and openness. We implement both reduction kernels, publish against an open PDK, and kept the coefficient array off chip, while other designs use proprietary PDKs, keep the coefficient array and polynomials on chip. However, this is the direct consequence of building for a shuttle tile rather than implementing the full scheme.

VI. CONCLUSION AND FUTURE WORK

We built an ML-KEM-512 NTT and inverse NTT accelerator, verified it against the FIPS 203 definition and the NIST

TABLE VIII
COMPARISON WITH PRIOR ML-KEM NTT ACCELERATORS.

	This work	Dam et al. [10]	KNightCore [12]
Platform	SKY130 ASIC	180 nm ASIC	Artix-7 FPGA
Scope	NTT engine	full RISC-V SoC	NTT engine
Area	52,731 μm^2	3.8025 mm ²	650 LUT, 130 registers
Frequency	50 MHz	118 MHz	not reported
Power	2.62 mW	307.9 mW	58 mW
Reductions	Montgomery, Barrett	not stated	Barrett
Coefficients	off chip	on chip	on chip (BRAM)
PDK	open	proprietary	vendor FPGA

ACVP vectors, demonstrated it on two FPGAs, and hardened it for the SKY130 open PDK on a Tiny Tapeout shuttle tile. The finished engine carries both Montgomery and Barrett reduction on one datapath and holds the 128 entry twiddle table in silicon, in 6067 standard cells at 72.7% core utilization on a 2 by 2 tile, drawing 2.62 mW at 50 MHz.

In the end, three results are worth separating from the numbers.

Constants and state cost very different amounts in silicon. The twiddle table is 1536 bits and occupies roughly 463 cells, because a constant is a fixed function of its index and the synthesizer minimizes it into gates, yet the coefficient array is the same order of magnitude in bits and does not compress at all, because it is mutable. This asymmetry is what decided which of the two we kept on chip.

Additionally, the two reduction kernels turned out to be complementary rather than interchangeable. Montgomery suits the butterfly multiply because a product can absorb the radix factor, while Barrett suits the inverse transform’s sum path because that path carries no product. Building both cost us 22% of our cell count and saved one multiply per inverse butterfly.

Least expected was how our FPGA results did not predict our ASIC results. Every architectural decision that looked free on an FPGA, a combinational multiplier and later a combinational reducer, became the binding constraint on SKY130nm, which required us to consider a pipelined approach that would meet timing specs.

Several directions remain open. The sequencer currently issues one butterfly at a time and the multiplier is busy for every one out of four clock cycles, so overlapping butterflies within a block would give roughly a four fold speedup. Within a block, the addresses that iteration i writes and iteration $i + 1$ reads never alias, so the hazard analysis is straightforward. In addition, the byte serial interface is the throughput limit rather than the arithmetic, so a wider interface, or one clocked on both edges, would help more than any change to the datapath. Lastly, the design currently supports ML-KEM-512, so since the modulus, the degree, and the primitive root are shared across all three ML-KEM parameter sets, extending it to the other security levels like 256 and 1012 requires changes to the host sequencing rather than to the hardware.

In conclusion, we hope that by publishing our work done on an NTT/INTT accelerator for ML-KEM 512, students and

educators can take inspiration from our work to either use it for educational purposes or improve upon it. We believe that by introducing the academic community to the Tiny Tapeout learning environment, we can empower newcomers to pursue a career in the ASIC design field.

ACKNOWLEDGMENT

One thing that we learned during this project is regardless of the theoretical understanding of the algorithm and functions that need to be performed, there still remains a large gap that we must bridge with the implementation on hardware. Our RTL code looked proper in simulation and in synthesis, but when running implementation, the place and route were the beasts to tame, and much of our effort was spent in debugging. For myself, it was a very large learning experience as implementing an algorithm of this scale was a first to me, while also trying to create it onto the Tiny Tapeout shuttle, and even if the mathematical logic of the algorithm is correct in the code, the entire design fails if there are issues with memory flow or logic gate mapping, which requires unique problem solving skills different to correcting mathematical logic.

We would like to acknowledge Matthew Venn, the creator of Tiny Tapeout, for creating the open-source environment that allows students, hobbyists, and even industry professionals to tapeout their own ASICs, providing a great opportunity for those that want to learn more about chip design or pursue a career in this field. We would also like to acknowledge Bryce Hackel, our mentor, and the rest of the SDUTC team who have helped us in our project.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” Federal Information Processing Standards Publication (FIPS) 203, Aug. 2024.
- [2] National Institute of Standards and Technology, “ACVP ML-KEM JSON specification,” Automated Cryptographic Validation Protocol. [Online]. Available: <https://github.com/usnistgov/ACVP-Server>
- [3] SkyWater Technology and Google, “SKY130 open source PDK.” [Online]. Available: <https://github.com/google/skywater-pdk>
- [4] Tiny Tapeout. [Online]. Available: <https://tinytapeout.com>
- [5] The LibreLane contributors, “LibreLane: an open source RTL to GDSII flow.” [Online]. Available: <https://github.com/librelane/librelane>
- [6] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [7] W. M. Gentleman and G. Sande, “Fast Fourier transforms: for fun and profit,” in *Proc. AFIPS Fall Joint Computer Conference*, vol. 29, 1966, pp. 563–578.

- [8] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [9] P. Barrett, "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *Advances in Cryptology (CRYPTO '86)*, Lecture Notes in Computer Science, vol. 263. Berlin, Germany: Springer, 1987, pp. 311–323.
- [10] D. T. Dam, K. D. Nguyen, D. H. Le, and C. K. Pham, "Accelerating post-quantum cryptography: A high-efficiency NTT for ML-KEM on RISC-V," *Electronics*, vol. 15, no. 1, art. 100, 2026.
- [11] D. E. S. Kundi, J. M. Bermudo Mera, P. Y. Strub, and M. Hutter, "High-performance NTT hardware accelerator to support ML-KEM and ML-DSA," in *Proc. ACM Workshop Attacks Solutions Hardware Security (ASHES)*, Salt Lake City, UT, USA, 2024.
- [12] B. Taghavi, R. Azarderakhsh, and M. Mozaffari Kermani, "KNightCore: An ultra-lightweight NTT-based polynomial multiplier for ML-KEM on resource-constrained platforms," in *Proc. Workshop Quantum-Resistant Security (QRSEC)*, Taipei, Taiwan, 2025.
- [13] Chang, Brendon, and Boudames, Marc, "ML-KEM-512 NTT/INTT Accelerator," Public code repo. [Online]. Available: <https://github.com/bdawg4113/ntt-butterfly-sky26c>